Chapter 9

EULER OPERATORS

In the architecture of GWB, the Euler operators have a central role. Based on the theory of plane models of Chapter 3, this chapter takes a closer look on the manipulation of boundary data structures, derives Euler operators, and presents rigorously their properties.

9.1 MANIPULATION OF PLANE MODELS

As noted in Section 6.3, the topological integrity of a boundary data structure of the winged-edge type can be enforced just by structural means. Nevertheless, the problem of manipulating a complicated data structure while making sure that all required references to nodes are maintained properly is somewhat difficult, and makes the writing of nontrivial algorithms (such as Boolean set operations) inconvenient.

Fortunately, the theory of plane models will give us help. As discussed in Section 3.5, plane models give us a useful mathematical abstraction of boundary models. In particular, the topological properties of the surface can be evaluated from the plane model. What we still need, however, is a scheme for creating plane models that have some desired properties.

Our aim here is to identify a small set of plane model manipulation operations that are powerful enough to describe *all* plane models of physical significance. While being general, the operations should be *safe* in that, for instance, nonorientable models (such as the Klein bottle of Figure 3.8 on page 42) cannot be created with them.

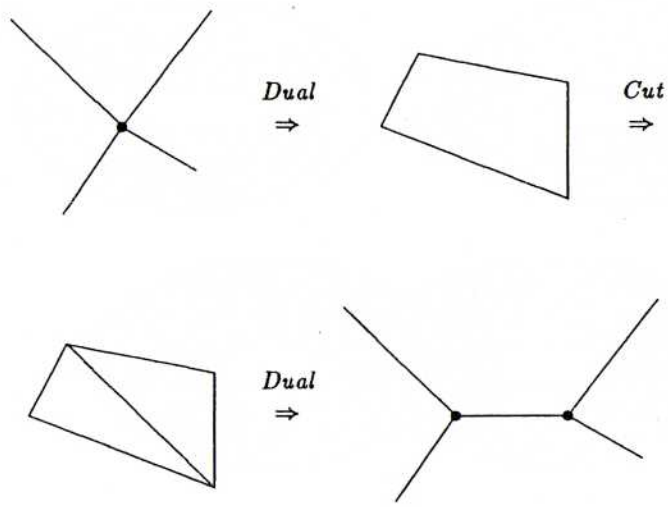

Figure 9.1 The vertex splitting operation.

9.1.1 Local Topological Operations

One of the main results of the plane model theory presented in Section 3.5 is the invariance theorem (Theorems 3.11 and 3.12, pages 43–45). The theorem states that the Betti numbers and all the topological properties conveyed by them (such as the Euler characteristic and orientability) remain invariant for all plane models that can be constructed for a surface.

A direct consequence of the invariance theorem is the following:

Lemma 9.1 *The topological properties of a plane model are not altered by either cutting a polygon in two or pasting together two polygons along a common edge.*

We interpret the cutting and pasting operations of polygons as a pair of inverse manipulation operations for plane models. That is, the cutting operation can divide a polygon into two polygons by joining two of its vertices with a new edge. Hence the sequence of edges around the polygon is divided into two sequences. Conversely, the inverse pasting operation removes an edge separating two polygons and merges their edge-cycles.

It is natural to ask whether there are other useful manipulation operations. In the search, the duality introduced in Section 3.5.8 offers help. To

see how, let us consider the effect of the cutting operation on the dual plane model. As demonstrated in Figure 9.1, a cutting operation on a polygon of the dual model has the effect of splitting the corresponding vertex of the original model in two vertices. Similarly, a pasting operation on the dual has the effect of a vertex joining operation that combines two vertices connected with an edge into one, and merges their edge-cycles. The splitting and joining operations are inverses of each other.

By Lemmas 9.1 and 3.13 of Section 3.5.8, a polygon cutting operation or its inverse on the dual plane model will preserve all topological characteristics of a plane model. Therefore, the vertex splitting operation or its inverse do not alter the topological properties of the plane model they modify, and they can be added to our set of manipulation operations.

Do we need other operations? An answer to this is provided by the following theorem:

Theorem 9.2 *Let PM be a realizable plane model of genus $= 0$ (i.e., a plane model topologically equivalent to a sphere). Then the polygon pasting and vertex joining operations are capable of reducing PM to a plane model that has just one vertex and one polygon, but no edges.*

Proof: *Suppose there is an edge E of PM such that it cannot be removed with either a polygon pasting or a vertex joining operation. Therefore, E must occur twice at the same polygon, and twice at the same vertex. Hence, E forms a closed curve on the 2-manifold M modeled by PM that does not split M into two components, and PM must be of genus > 0.*[1] *By contradiction, no such E can occur, and all edges of PM can be removed.*

So, by means of the pasting and joining operations only, *all* plane models of the sphere can be reduced to a one-vertex, one-polygon model. We call this special model the *skeletal plane model.* A sample case of the theorem is presented in Figure 9.2. In this particular case, all vertices of the object are first made to occur in a single polygon by removing edges with the pasting operations until no further faces can be merged (Figure 9.2(a,b)). Thereafter, all remaining edges are removed with the joining operation until only the skeletal plane model (Figure 9.2(c)) remains.

Of course, the skeletal plane model as such is not a particularly useful model of a sphere or a cube. Its significance for our purposes is due to the following:

Corollary 9.3 *Let PM be a realizable plane model of genus 0. Starting from a plane model consisting of a single vertex and a single polygon, PM*

[1] See discussion on the first Betti number h_1 on page 45.

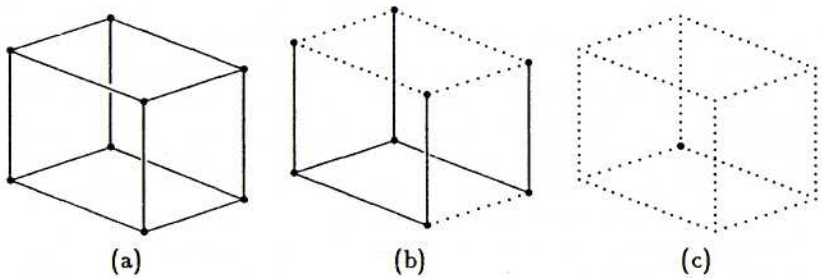

Figure 9.2 Reducing a cube to the skeletal plane model.

can be created by a sequence of e polygon cutting or vertex splitting operations, where e is the number of edges of PM.

Proof: *By Theorem 9.2, all edges of PM can be removed with a sequence S consisting of e pasting and joining operations. Because the polygon cutting and the vertex splitting operations are the exact inverses of the pasting and joining operations, the sequence S^{-1} consisting of the cutting and splitting operations corresponding with the operations of S in reverse order is a sequence satisfying the claim.*

That is, starting from the skeletal plane model, *all* plane models of genus 0 can be created by means of the cutting and splitting operations! Therefore, in order to create an arbitrary plane model of genus 0, we only need to supply the "prototype" skeletal plane model as a primitive, and work on it with the two operations.

As the two manipulation operations do not alter the global topological properties of the plane model they operate on, we call them *local topological operations.*

9.1.2 Global Topological Operations

In addition to the local topological operations, we need manipulation operations which are capable of producing objects whose genus is greater than zero. As these operations must definitely alter the global topological properties of a plane model, we call them *global topological operations.*

At this point a slight diversion back to the topology of surfaces is necessary. Given two surfaces, it is always possible to construct a new surface by cutting a small disk off from each original surface, and pasting the two

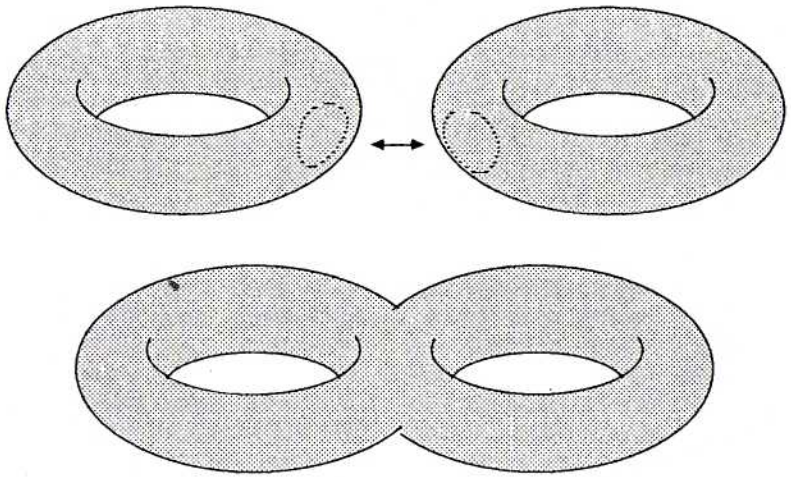

Figure 9.3 Connected sum of two tori.

surfaces together along the boundary of the cutouts. For instance, the *two-holed torus* can be created in this fashion from two tori (see Figure 9.3).

The surface resulting from the cut-and-paste procedure is called the *connected sum* of the two original surfaces. That the cut-and-paste procedure is general enough for creating all surfaces we are interested in, is shown by the so-called *classification theorem*:

Theorem 9.4 *Every orientable surface is topologically equivalent either to the sphere, or the connected sum of n tori.*

The proof of the classification theorem is beyond the scope of this text; again, see [52] for a good exposition.

The cut-and-paste procedure can be readily interpreted in the language of plane models. In the interpretation, the "cutting of disks" becomes the creation of two polygons in the two plane models at the desired positions. The "pasting" is modeled as an operation that combines those two polygons with a new edge that will occur twice at the new combined polygon.

For instance, Figure 9.4 depicts the plane models of a cube and a tetrahedron. They can be combined by joining the polygon p of the cube and the "external" polygon with a new edge. Observe that new edge occurs twice in the combined polygon p'; i.e., p' now has nine edges.

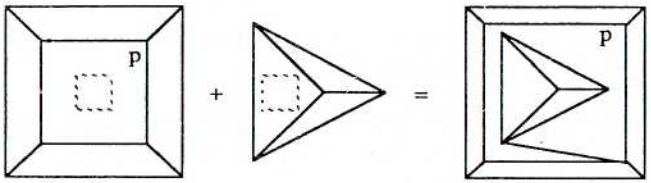

Figure 9.4 Connected sum of two plane models.

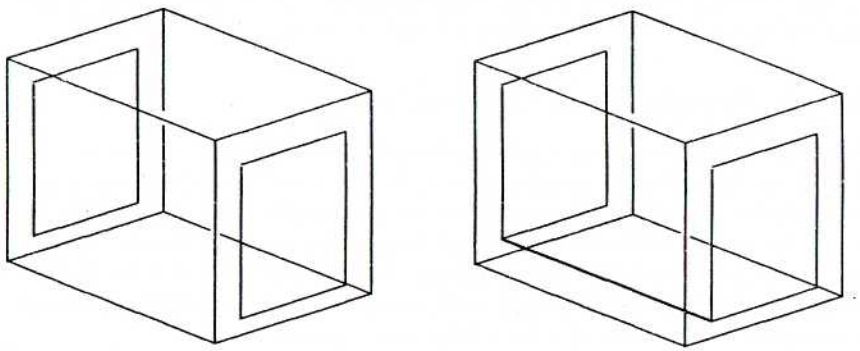

Figure 9.5 Connected sum on a single plane model.

If we interpret the two original plane models of Figure 9.4 as a single disconnected plane model, we may observe that the connected sum operation alters the connectivity of the model. By extending the use of the connected sum operation to two polygons of a connected (component of) plane model, we can also alter the genus of a plane model, as desired. For instance, we can produce a hole through the cube of Figure 9.5 by joining the two polygons at its ends with a new edge.

Again, the connected sum operation has a natural inverse operation. This *connected minus* operation can be used to remove an edge occurring twice at a polygon. As a result, the polygon is divided into two polygons.

Furnished with the connected sum operation and its inverse, we can show the following:

Theorem 9.5 *Let PM be a realizable plane model. Using the local topological operations (polygon cutting and pasting, vertex splitting and joining), and the connected sum and connected minus operations, all edges of PM can be removed, i.e., PM can be reduced to the skeletal plane model.*

Proof: *Consider an arbitrary edge E of PM. The following case analysis applies:*

1. *E separates two distinct polygons. In this case, E can be removed with the polygon pasting operation.*
2. *E occurs in a single polygon between two distinct vertices. In this case, E can be removed with the vertex joining operation.*
3. *Otherwise, E can be removed with the connected minus operation.*

Because all cases are covered, all edges can be removed.

Similarly to the corresponding results of Theorem 7.2 and Corollary 7.3, the following corollary is readily seen to be valid:

Corollary 9.6 *Let PM be a realizable plane model. Starting from the skeletal plane model, PM can be created by a finite sequence of local topological operations and connected sum or minus operations.*

Proof: *Analogous to Corollary 9.3.*

That is, *all* realizable plane models can be created with the local topological operations and the connected sum and minus operations.

9.1.3 Soundness of Plane Model Operations

Corollary 9.6 tells us that every plane model of interest to us (that is, all realizable plane models) can be generated by means of the plane model manipulation operations. Therefore, the operations are *complete*, i.e., they are powerful enough for describing all realizable plane models.

It is natural to ask whether the operations also are *sound*, i.e., whether we can be certain that no others besides the realizable plane models can be generated through their use. That this really is so is shown by the following theorem:

Theorem 9.7 *Both the local plane model manipulation operations (polygon cutting and pasting, vertex splitting and joining), and the global plane model manipulation operations (connected sum and connected minus) are* sound, *i.e., they cannot create nonrealizable plane models.*

Proof: *Let us consider an arbitrary finite sequence of plane model manipulation operations. The proof will be inductive on the length of the sequence.*

First, the criteria for realizable plane models as given by the Definitions 3.9 and 3.10 of Chapter 3 are as follows:

1. Edge identification: *Every edge of the plane model is topologically identified with exactly one other edge.*
2. Cyclical identification: *The polygons identified at each vertex can be arranged in a cycle such that each consecutive pair of polygons in the cycle is identified at an edge adjacent to the vertex.*
3. Orientability: *The polygons of the model can be oriented so that for each pair of identified edges, one edge occurs in its positive orientation in the direction of its polygon, and the other in the negative orientation.*

As the basis of the induction, the skeletal plane model trivially satisfies the three conditions. From there on, we have exactly six cases corresponding to the six possible operations to consider. In each case, we shall have to show that the operation will preserve the criteria 1–3 stated above.

Let us only elaborate two of the cases here and leave the rest as an exercise to the reader. First, the vertex splitting operation adds a pair of identified edges that occur in opposite orientations in their respective polygons in the plane model. Hence, if the plane model originally satisfies the edge identification and the orientability constraints, it will do so also after the operation. As for the cyclical identification constraint, observe that the vertex splitting operation divides the cycle of edges of a vertex into two sequences; hence also cyclical identification will remain unaltered.

As vertex splitting and face cutting are duals of each other, the reasoning above immediately has the consequence that the face splitting operation will not alter the validity criteria either. Finally, also the connected sum operation can be shown to preserve the criteria.

Because none of the cases violates the inductive hypothesis, an arbitrary sequence of the operations will create a model that satisfies the realizability criteria, and we have the theorem.

Together, Corollary 9.6 and Theorem 9.7 state that the local and global manipulations are *sound and complete* for the family of realizable plane models. Therefore, they fill exactly the desired characteristics stated in the start of this section.

9.2 MANIPULATION OF BOUNDARY MODELS

As seen in the previous section, a small collection of manipulation operations (and their inverses) are sufficient for describing all plane models of

interest. According to the theory, only three basic types of manipulative operations were needed:

1. A "prototype" primitive that creates skeletal plane models.
2. Two local topological operations that subdivide the sequences of edges of a face or of a vertex.
3. A global topological operation that implements a connected sum of two polygons (either from distinct models or one model).

The theory carries over to boundary models. The manipulation operations on plane models will have their counterparts in the form of *Euler operators* that act on the topology of a boundary model data structure. Being realizations of the theoretical plane model operations for the domain of real bounary data structures, Euler operators share their completeness and soundness stated as Corollary 9.6 and Theorem 9.7.

9.2.1 Notation and Conventions

Euler operators were originally introduced by Baumgart [13,14] in the context of the winged-edge data structure. As this data structure is the natural basis for discussing the operators, we shall assume the use of the data structure of Figure 6.6, extended with loop nodes for representing nonsimple faces. For convenience of language, the internal loops will be termed *rings*. We shall also use the term *shell* introduced in Section 3.5.6 to denote a connected component of a boundary data structure.

We shall also use the convention of having *empty loops* in the data structure, i.e., loops consisting of just a single "lone" vertex with no edges at all. Such entities are useful, for instance, for representing the boundary model counterpart of the skeletal plane model.

By historical convention, the operators are usually denoted by rather cryptic, mnemonic names. The key to the names to be used in this text is given in the below:

M — make	V — vertex	H — hole
K — kill	E — edge	R — ring
S — split	F — face	
J — join	S — solid	

For instance, the name "mev" translates as "Make Edge, Vertex".

It should be borne in mind that Euler operators can be implemented in several ways on top of widely varying data structures. We shall emphasize this by using the SMALL CAPS font when refering to an Euler operator

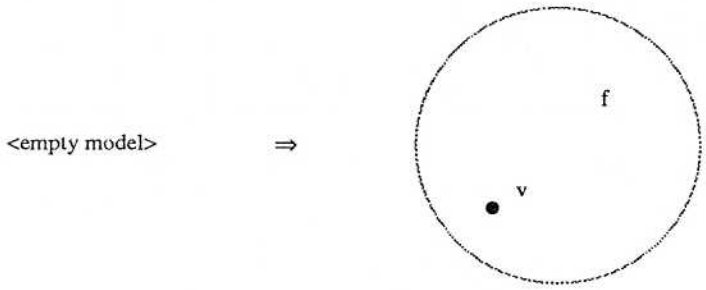

Figure 9.6 The MVFS operator.

"in general," without regard for a particular implementation. This is in contrast to our usual convention of using the `typewriter` font when refering to an actual piece of code.

9.2.2 Skeletal Primitives: MVFS and KVFS

The theory tells us that by means of local and global topological manipulations, all plane models of interest can be created from a single skeletal plane model. This result means that a single skeletal primitive is sufficient for us.

The operator MVFS takes this role in our collection. As the name suggests, it creates from scratch an instance of the data structure that has just one face and one "lone" vertex. Hence the new face has one "empty" loop with no edges at all. The effect of MVFS is depicted in Figure 9.6.

The "solid" created by MVFS may not satisfy the intuitive notion of a solid object. Nevertheless, it it useful as the initial state of creating a boundary model with a sequence of Euler operators.

Similarly to the plane model manipulation operations, all Euler operators will have corresponding inverse operators that can undo the effect of the "positive" operator. The inverse of MVFS is called KVFS, and it destroys a skeletal instance of the data structure identical to that created by MVFS.

9.2.3 Local Manipulations

In analogy to local topological operations on plane models, we need operators that can work on the local topological properties of a boundary model.

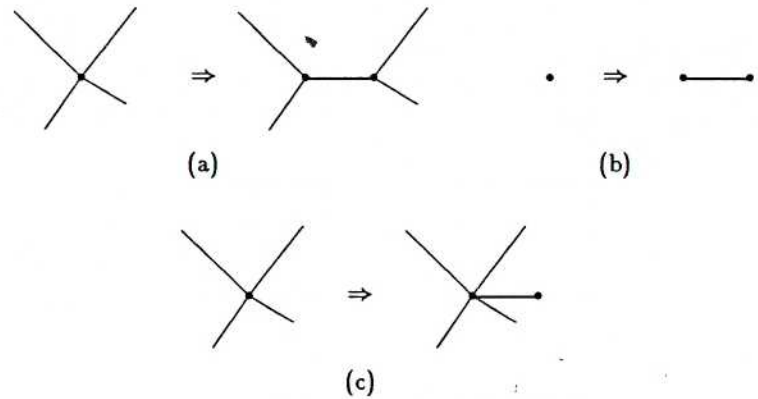

Figure 9.7 The MEV operator.

MEV, KEV

Operators MEV and MEF (to be described in the next section), with their inverses (KEV, KEF) correspond exactly with the polygon and vertex splitting operations for plane models. Hence MEV subdivides the cycle of edges of a vertex into two cycles by "splitting" a vertex into two vertices, joined with a new edge (see Figure 9.7(a)). The net effect of this is to add one vertex and one edge in the data structure, as suggested by the name of the operator.

The applicability of MEV is extended to "lone" vertices (i.e., vertices appearing in empty loops such as the one created by MVFS) by considering the result of subdividing a vertex with no edges at all as consisting of two vertices joined with an edge (see Figure 9.7(b)). We also include the case that a new vertex joined with the old vertex by means of a new edge is created as depicted in Figure 9.7(c).

The inverse operator KEV is capable of undoing any of the three cases of Figure 9.7. In other words, given an edge connecting two distinct vertices, KEV can remove the edge, collapse the vertices into one and merge their edge cycles.

MEF, KEF

The operator MEF subdivides a loop by joining two vertices with a new edge. Its net effect is to add one new edge and face into the data structure.

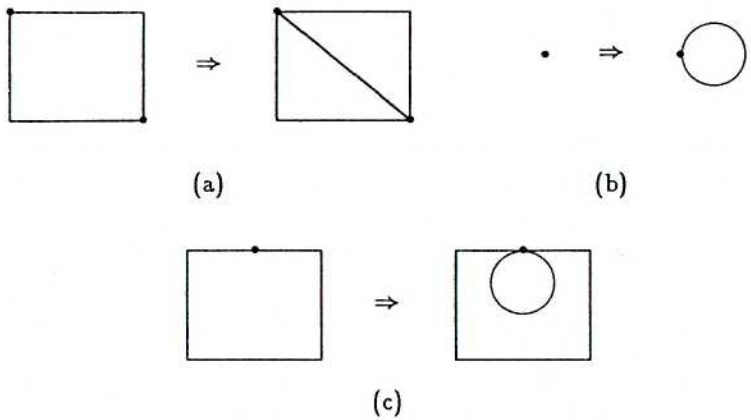

Figure 9.8 The MEF operator.

In addition to the ordinary case of Figure 9.8(a), we also extend the applicability of MEF to empty loops in strict analogy with MEV. Hence the result of subdividing a "lone" vertex consists of a "circular" edge separating two faces. Observe that the starting and final vertices of such an edge are equal; this is a case admitted by the winged-edge data structure (Figure 9.8(b)). As a more general case of this, it is always possible to "connect" a vertex to itself by means of MEF as depicted in Figure 9.8(c).

Again, the inverse operator KEF can undo the effect of MEF in each case. More accurately, given an edge adjacent to two distinct faces, KEF is capable of removing the edge, and joining the two faces into one whose bounding loop is the result of merging the two original boundaries.

The reader is urged to contemplate Figures 9.7 and 9.8 until their analogy is clear to him. Note also how the descriptions of KEV and KEF are "duals" to each other.

KEMR, MEKR

Operators MEV and MEF are "fundamental" in that their significance can be derived directly from the theory of plane models. In contrast, the remaining local manipulation is a "convenience" operator whose necessity is due to our conventions rather than the underlying theory.

In the above, "empty loops" were used to realize the skeletal plane mode in the boundary data sturcture domain. To make full use of empty loops, it is useful to introduce an operator specially tailored for their creation.

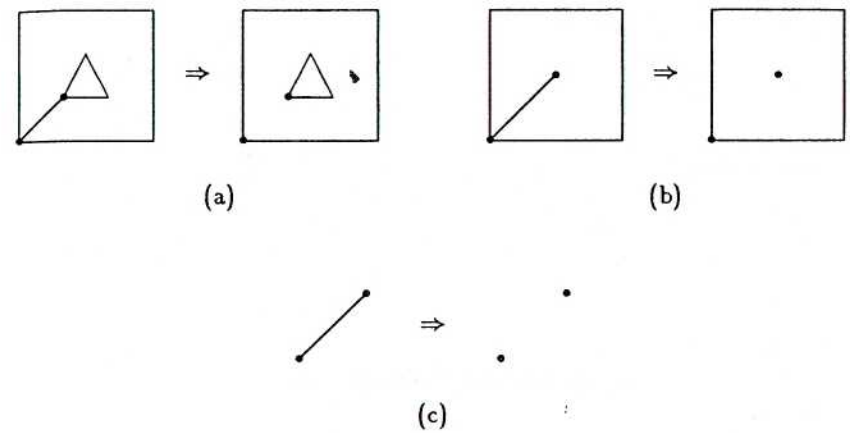

Figure 9.9 The KEMR operator.

The resulting operator KEMR splits a loop into two new ones by removing an edge that appears twice in it (see Figure 9.9(a)). Hence KEMR divides a connected bounding curve of a face into two bounding curves, and has the net effect of removing one edge and adding one ring to the data structure. The special cases that one or both of the resulting loops are empty are also included (Figure 9.9(b, c)).

The inverse operator MEKR can merge two loops of a face by joining one vertex of each with a new edge.

9.2.4 Global Manipulations: KFMRH, MFKRH

None of the operators discussed so far is capable of modifying the global topological properties of the data structure e.g. by dividing a solid into two components or by creating a "hole." This is the task of the remaining Euler operator.

Operator KFMRH is a realization of the connected sum operation discussed in Section 9.1.2. Given two faces (say, f_1 and f_2), it joins them into one face by transforming the bounding loop of f_2 to a ring of f_1. Hence its net effect is to remove one face (f_2) and add one ring. Note that KFMRH has no effect on the local arrangement of edges and vertices of its argument faces—it is truly a global manipulation. This also means that it is hard to illustrate its effect. One attempt is given in Figure 9.10(a).

KFMRH is actually a misnomer, because the operator does not neces-

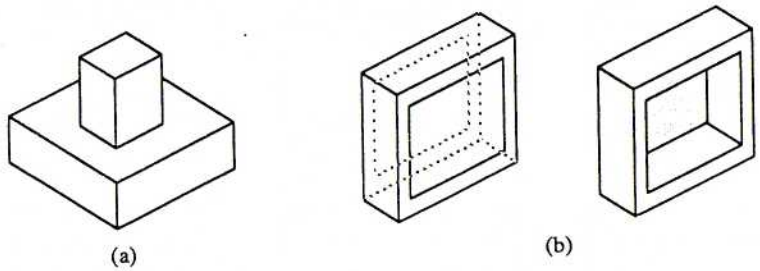

Figure 9.10 The KFMRH operator.

sarily create a "hole". Actually, KFMRH creates a hole only if the two argument faces belong to the same shell. When applied to faces belonging to two distinct shells, its effect is to combine them into one shell, and the name "KFSMR" (where S now stands for "shell") would seem a better condensed description of what really takes place (see Figure 9.10(b)). But neither of the two names is better than the other, and we shall stick to KFMRH.

Again, an inverse operator MFKRH is capable of reversing the effect of KFMRH. It modifies a ring of a face as the bounding loop of a new face.

Recall that the connected sum operation for plane models was specified to add a new edge connecting the two polygons to be joined, while KFMRH achieves the same effect by making the boundary of one face a ring in the other. Otherwise, the operations are strictly analogous.

9.3 AN EXAMPLE OF EULER OPERATORS

Let us clarify the use of Euler operators by means of a simple example, the construction of a rectilinear box with one rectilinear hole. The construction is depicted in Figure 9.11 in terms of both plane models and three-dimensional space models.

The construction starts by creating the skeletal model by means of a MVFS (a). By applying MEV three times, three edges and vertices are added so as to create (b). By joining the end vertices of the edge string (b) with a MEF, the model (c) is created. Note that the space model of (c) consists of two planar faces tightly upon each other like the two sides of a piece of paper. Such a model is called a *lamina*.

Four MEV's lead us to the model (d) that has the side edges of the emerging box. Their tips are joined with four MEF's to form the side faces

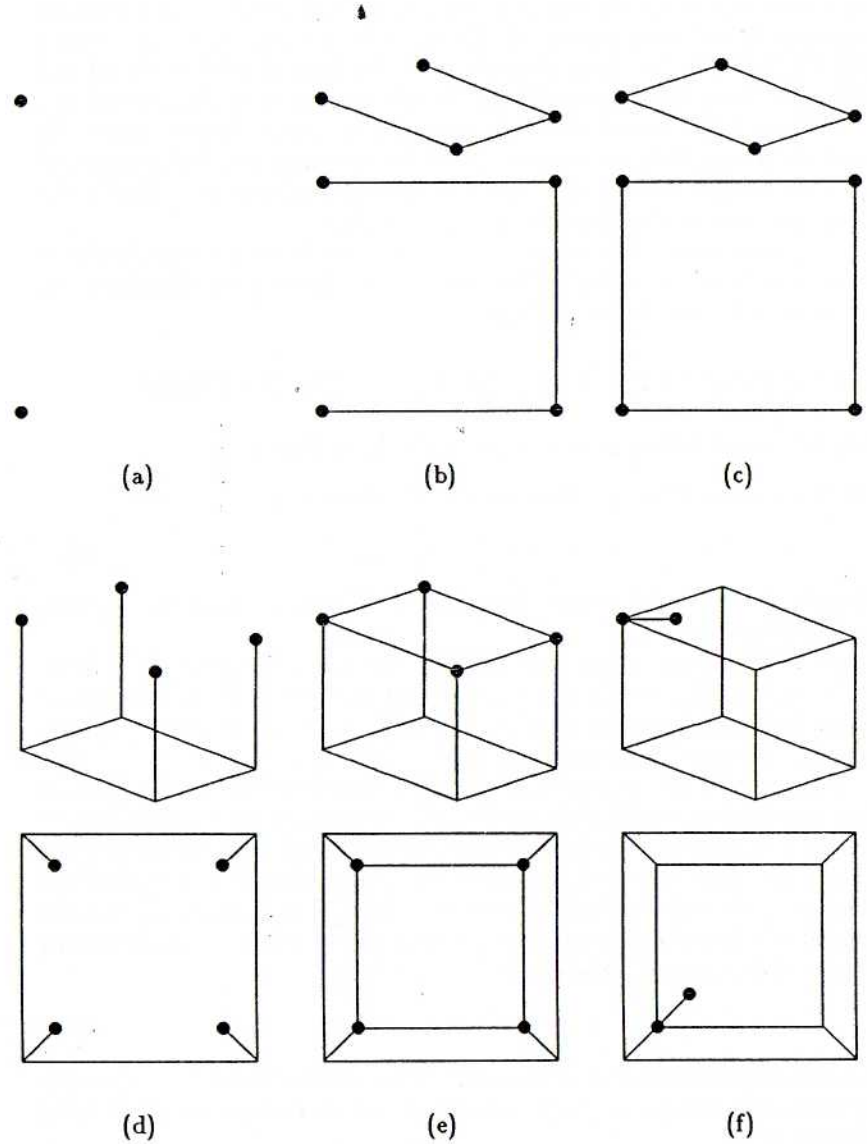

Figure 9.11 Example of Euler operators.

(e). We now have formed the model of a simple box.

To create the hole, we first need to form an interior face on the top of the box (e). We do that by adding an edge on the top face with a MEV (f), and removing the edge with a KEMR (g). This gives us an empty loop with one vertex only inside the top face. After three additional edges are drawn from that vertex with MEV's (h), the interior face can be formed with a MEF (i). Now the side faces of the hole can be formed with MEV's (j) and MEF's (k). Steps (h) through (k) are strictly analogous to (b) through (e).

We now have formed all edges and vertices of the desired result. To finish the description, the bottom face of the emerging hole is "subtracted" from the bottom face of the box by means of a KFMRH (l). This is the same operation as that depicted in Figure 9.10(a).

Of course, to actually create the box, we need to assign coordinates to vertices as they are created. This can be accomplished by operators that create vertices (i.e., MVFS and MEV).

9.4 PROPERTIES OF EULER OPERATORS

9.4.1 Euler-Poincaré Formula Revisited

Recall the Euler-Poincaré Formula 3.2 of Section 3.5.6:

$$v - e + f = 2(s - h), \tag{9.1}$$

where v, e, f, s, and h denote the numbers of vertices, edges, faces, shells, and holes in a solid.

From the theory of plane models it it clear that a collection of faces, edges, and vertices can be a valid boundary model only if the numbers of these elements satisfy Equation 9.1. Hence it can be considered a necessary integrity criterion for boundary models.

Equation 9.1 can be modified to make it consistent with our boundary data structure conventions by introducing a new parameter r, standing for the number of rings. Obviously, each ring can be removed by introducing a new edge that connects it into the external bounding loop of its face (see Figure 6.7). No new vertices or faces are added in this operation, but the number of edges after the removal of r rings will be $e' = e + r$. Substituting e' for e in Equation 9.1 gives us the desired form

$$v - e + f = 2(s - h) + r \tag{9.2}$$

that describes a necessary condition for the numbers of elements of a boundary data structure (v, e, f, r), and the global characteristics of the solid modeled (s, h).

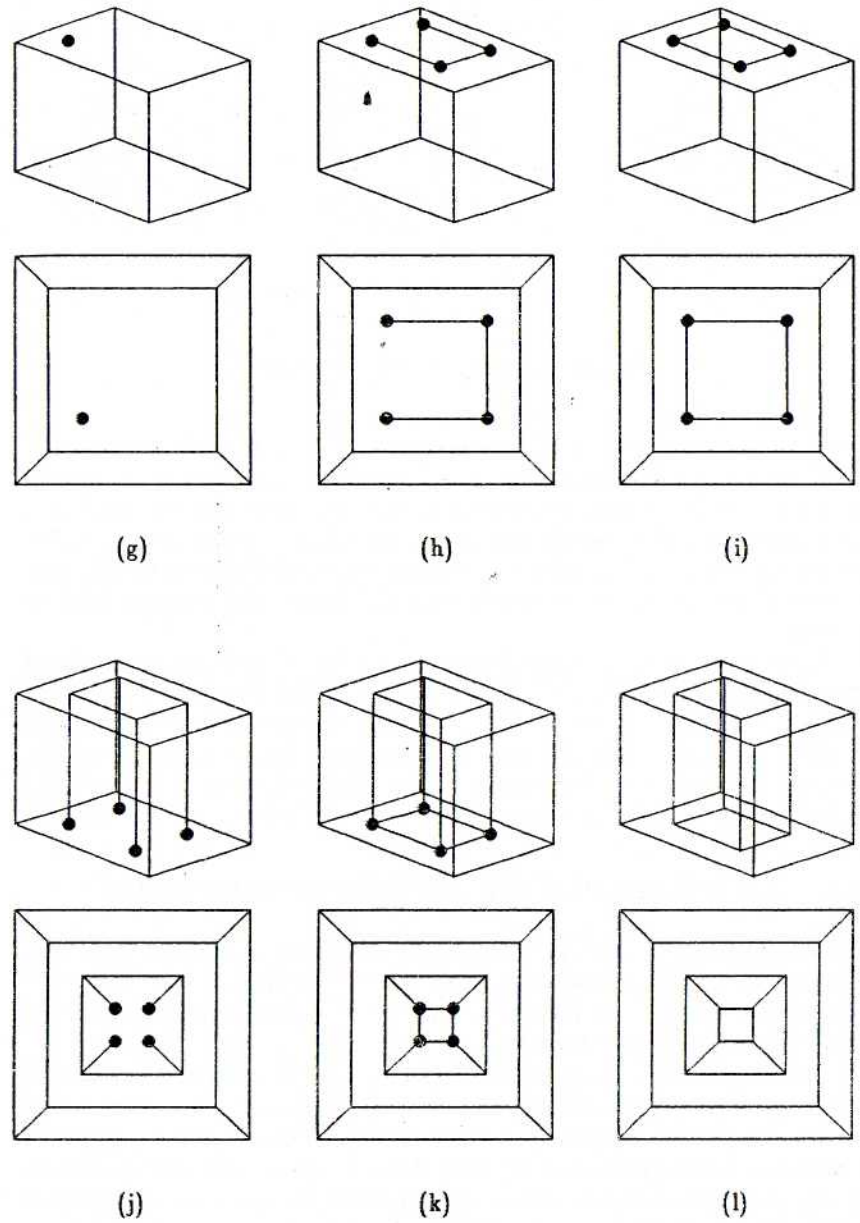

Figure 9.11 Example of Euler operators (cont.).

9.4.2 Algebraic Properties of Euler Operators

Observe that the data structure created by MVFS has $v = f = s = 1$ and $e = h = r = 0$. By substituting these numbers into Equation 9.2, we can see that they make it hold. Furthermore, all other operators will add or remove entities of the data structure in a fashion that keeps the condition expressed by Equation 9.2 satisfied. For instance, MFKRH adds one face and removes a shell and a ring. In terms of Equation 9.2, its effect is to replace f by $f' = f + 1$, r by $r' = r - 1$, and h by $h' = h - 1$. By substituting f', r', and h' into 9.2 one can readily see that the formula will remain satisfied. Hence Euler operators have the property that *the numbers of elements in each data structure created by them will satisfy the Euler formula of Equation* 9.2.

This observation suggests that Euler operators could be analyzed in purely algebraic terms. Following Braid et al. [19], let us consider a six-dimensional discrete space, and call its axes v, e, f, h, r, and s. Then Equation 9.2 can be interpreted as the equation of a five-dimensional hyperplane E of the space. In this view, the analogy of a boundary model is a point

$$P = (v\ e\ f\ h\ r\ s),$$

of the six-dimensional space and the condition of Equation 9.2 can be interpreted as stating that "interesting" points of the space are those lying on the plane E.

The hyperplane E is itself a five-dimensional discrete subspace of the six-dimensional space. Therefore, it can be spanned by five six-vectors V_i satisfying the following conditions:

1. Each vector V_i lies on E.

2. Vectors V_i are linearly independent.

After such a set of base vectors of E has been chosen, all points of E can obviously be expressed as linear combinations of the base vectors.

In the analogy suggested above, Euler operators take the role of the base vectors. In this view, our collection would be interpreted as the set of base vectors summarized in Table 9.1. Obviously, an infinite amount of other sets of base vectors are possible, and every set can be interpreted as a particular collection of Euler operators. In practice, it is desirable to choose simple operators that correspond with "short" vectors of E, and the published collections of Euler operators [14,33,34,19,4,78,75] are almost identical with each other.

This algebraic view of Euler operators can be applied to gain interesting insight into their properties. To see how, let us arrange the base vectors

Operator	*Base Vector*					
	v	e	f	h	r	s
MEV	1	1	0	0	0	0
MEF	0	1	1	0	0	0
MVFS	1	0	1	0	0	1
KEMR	0	−1	0	0	1	0
KFMRH	0	0	−1	1	1	0
KEV	−1	−1	0	0	0	0
KEF	0	−1	−1	0	0	0
KVFS	−1	0	−1	0	0	−1
MEKR	0	1	0	0	−1	0
MFKRH	0	0	1	−1	−1	0

Table 9.1 Euler operators.

corresponding with the Euler operators of our collection, together with the coefficients of Equation 9.2 into the matrix M given below:

$$M = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 1 & 1 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 1 & -1 & 1 & 2 & -1 & -2 \end{bmatrix} \tag{9.3}$$

Observe that the coefficient vector on the last row of M acts as the normal vector of the hyperplane E. Because Euler operators form a base of E, the matrix has an inverse. The inverse matrix M^{-1} is given below.

$$M^{-1} = \frac{1}{12} \begin{bmatrix} 9 & -5 & 2 & -2 & 3 & 1 \\ -3 & 5 & -2 & 2 & -3 & -1 \\ 3 & 7 & 2 & -2 & 3 & 1 \\ -6 & 2 & 4 & 8 & -6 & 2 \\ 3 & 5 & -2 & 2 & 9 & -1 \\ -6 & -2 & 8 & 4 & -6 & -2 \end{bmatrix} \tag{9.4}$$

The inverse matrix can be used to calculate the "Euler coordinates" of any $[v\ e\ f\ h\ r\ s]$-tuple. For instance, the object constructed in Figure 9.11 has $v = 16$ vertices, $e = 24$ edges, $f = 10$ faces, $r = 2$ rings, $h = 1$ hole, and $s = 1$ shell. Hence it corresponds with the location

$$P = [16\ 24\ 10\ 1\ 2\ 1]$$

of the discrete six-dimensional space. Now the product

$$P \times M^{-1} = [15\ 10\ 1\ 1\ 1\ 0]^T$$

tells us that to model this solid, a total of 15 MEV's, 10 MEF's, 1 MVFS's, 1 KFMRH's and 1 KEMR are needed at the minimum. That the last component is zero shows that the original 6-tuple satisfies Equation 9.2. Observe that the construction of Figure 9.11 uses exactly these numbers of operators; hence, it it "optimal" in the length of the construction.

The algebraic analysis gives us hence a mental tool for discussing the complexity of Euler operator descriptions of boundary models. As another example of this, if $s = 1$, the general product

$$[v\ e\ f\ h\ r\ s] \times M^{-1}$$

yields the following operator counts:

$$\begin{array}{ll} v-1 & \text{MEV's} \\ f+h-1 & \text{MEF's} \\ 1 & \text{MVFS} \\ h & \text{KFMRH's} \\ r-h & \text{KEMR's} \end{array} \tag{9.5}$$

Clearly, Equations 9.5 give the smallest numbers of operators needed to model an object corresponding with location $[v\ e\ f\ h\ r\ 1]$ of the six-dimensional discrete space.

Unfortunately, the inverse operators count as -1 in Equations 9.5, which makes their interpretation difficult. Nevertheless, because all models of interest have $v, f > 0$ and $f > h$, the four first counts are nonnegative. This suggests that all connected objects can be modeled without the "inverse" operators KEV, KEF, KVFS, and MFKRH, a fact that will be shown in Chapter 16. Also, from the fact that the last line can be negative, we can infer that some models cannot be created without the MEKR operator.

9.4.3 Descriptive Power of Euler Operators

Euler operators remove some part of the complexity of dealing with boundary data structures. Does this impose a penalty as far as the generality and flexibility is concerned? Is it possible to create arbitrary models under restriction that only Euler operators are used in their construction?

The algebraic view to Euler operators presented in the last section gives us some insight into these questions. Obviously, each 6-tuple $[v\ e\ f\ h\ r\ s]$ can be associated with a tuple of "Euler coordinates" through the inverse of the transition matrix M. But there is still some concern, however. As the

reader can easily verify with a counterexample, a 6-tuple does not uniquely determine a solid. Can we produce all objects that have the same tuple? Furthermore, while the discussion of the last section suggests that the number of operators needed to model a solid grows linearly with the complexity of the object, it is not completely clear whether an operator sequence of that length exists for all objects, because inverse Euler operators contribute as -1 to the counts given by the inverse transition matrix.

The theory of plane model manipulations presented in Section 9.1 resolves the first question. In particular, Theorems 9.5 and 9.7, and Corollary 9.6 can be reinterpreted in the domain of boundary data structures and Euler operators. In this new form, the theorems are as follows:

Theorem 9.8 *Let S be a valid boundary data structure (i.e., S satisfies all topological integrity constraints). Then there exists a finite sequence of Euler operators that can completely remove S.*

Corollary 9.9 *All valid boundary data structures can be created with a finite sequence of Euler operators.*

The direct proofs are analogous to those of Theorem 9.5 and Corollary 9.6. In Chapter 16, we shall give a constructive proof by developing an algorithm that actually constructs such a sequence of operators.

Theorem 9.10 *Euler operators are* sound, *i.e., they cannot create topologically invalid boundary data structures.*

Again, the direct proof is similar to that of Theorem 9.7. As an additional case, the soundness of KEMR and MEKR would have to be demonstrated.

Two notes on the soundness theorem may be needed. First, to model anything at all, a sequence of operators must satisfy certain "syntactic" validity criteria pertaining to a particular way of representing Euler operators. For instance, all entities that operators work on must exist and be of proper type. Say, the operator KEF can be applied only to an edge that appears in two distinct faces.

Second, the theorem has nothing to say on the *geometric* validity of the resulting model; by assigning inappropriate geometric information to faces, edges, and vertices, it is still perfectly possible to create invalid models that have no physical significance.

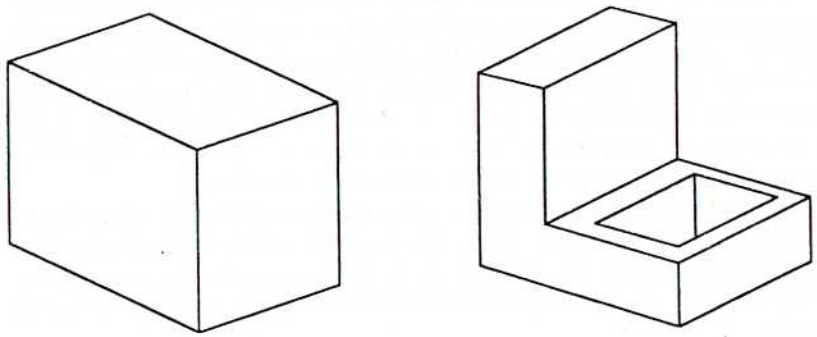

Figure 9.12 Sample solids.

PROBLEMS

9.1. Complete the proof of Theorem 9.7 on page 145.

9.2. Give an Euler operator construction in the style of Figure 9.11 for the following objects (see Figure 9.12):

(a) A rectilinear block with six faces and eight vertices.

(b) An "L-brick with hole" with 12 faces and 20 vertices.

9.3. Show that some solids cannot be described without the MEKR operator.

Hint: Equations 9.5 suggest that for such an object, $h > r$.

9.4. Provide an example of two different boundary models having the same $[v\ e\ f\ h\ r\ s]$-tuple.

9.5. Demonstrate the soundness of the Euler operators KEMR and MEKR.

Hint: You will have to show that they preserve the conditions of edge identification, cyclical identification, and orientability.

BIBLIOGRAPHIC NOTES

The idea of using plane models to argue rigorously about the formal properties of Euler operators was originally presented in the article [75] of the author. Most of the material presented in this chapter stems from this publication, but appears here in extended and clarified format.

The discussion on algebraic properties of Euler operators presented in Section 9.5.2 is based on the exposition of Braid et al. [19] with some minor changes in details.